Démonstrateur d'attaques contre un ML-NIDS

État de l'art et premier prototype réseau

Projet VAP SSR 2026–2027

Réunion avec Grégory Blanc

27 octobre 2026

Avancées depuis la réunion du 6 octobre

- **État de l'art rédigé** sur les attaques contre les ML-NIDS.
- **Premier site développé** pour construire une topologie réseau par glisser-déposer, dans l'esprit de Packet Tracer.

À faire ensuite

Choisir les modèles et les attaques, puis les intégrer au démonstrateur.

Rappel : les 8 papiers présentés le 6 octobre

Papier	**Modèle visé**	**Ce que fait l'attaque**
Han (2021)	KitNET (autoencodeurs)	Retarder et ajouter des paquets
Sharon (2022)	Autoencodeurs, Isolation Forest	Imiter le rythme du trafic normal
Okada (2025)	MLP	Modifier le TCP aux points clés
Mao (2025)	MLP, CNN, ResCNN, LSTM, attention	Tromper plusieurs modèles à la fois
Zhang (2022)	MLP, CNN, C-LSTM	Interroger le détecteur, ajuster
Venturi (2024)	E-GraphSAGE, LineGraphSAGE	Ajouter des communications
Venkatesan (2021)	SVM	Ajouter des exemples mal étiquetés
Das (2025)	SVM, Random Forest, MLP	Changer les étiquettes

6 papiers sur l'**évasion**, seulement 2 sur l'**empoisonnement**.

État de l'art : pourquoi trois nouveaux papiers ?

Au 6 octobre, l'empoisonnement reposait sur Venkatesan et Das :

- l'objectif était seulement de **dégrader** le détecteur (plus d'erreurs) ;
- l'attaquant était **déjà à l'intérieur** (accès aux données, contrôleur SDN compromis) ;
- les attaques portaient sur des **tableaux de caractéristiques**, pas sur des paquets.

Trois manques, trois papiers ajoutés :

1. Un attaquant **qui n'entre pas** chez la victime → **Alhussien**.
2. Les **portes dérobées** : le modèle semble bon, mais laisse passer le trafic qui porte un motif secret → **PCAP-Backdoor** et **Ma**.
3. Une attaque construite sur de **vrais paquets (PCAP)**, comme ceux capturés par notre prototype → **PCAP-Backdoor**.

État de l'art : ce que chaque nouveau papier apporte

	Qui est l'attaquant ?	Ce qu'il fait	Pourquoi on l'a retenu
Alhussien (2023)	Il pollue une **base de données publique** que la victime utilise pour entraîner son détecteur.	Il modifie 5 % de cette base. La détection des attaques passe de 79 % à 47 %.	Scénario **réaliste** : pas besoin de pirater la victime. Défense simple : plus de données internes.
PCAP-Backdoor (2026)	Il peut ajouter du trafic dans les **captures d'entraînement**.	Il glisse un motif dans du trafic normal, **sans changer les étiquettes**. Ses attaques avec ce motif sont ensuite classées « bénin ».	Travaille sur des **fichiers PCAP**, comme notre capture : le papier le plus proche du démonstrateur.
Ma (2025)	Le **prestataire** à qui la victime confie l'entraînement.	Il marque 0,1 % des exemples avec un motif et les étiquette « bénin ». Plus de 97 % des attaques marquées passent.	Montre qu'un **test classique ne suffit pas** (moins de 3 % de perte) et propose une **défense**.

À retenir : une porte dérobée ne se voit pas aux tests normaux, seulement avec le motif.

Récapitulatif : les 11 papiers de l'état de l'art

Évasion : tromper un modèle déjà entraîné

Empoisonnement : tricher pendant l'entraînement

Papier	Modèle visé	Ce que fait l'attaque	
Han (2021)	KitNET (autoencodeurs)	Retarder et ajouter des paquets	
Sharon (2022)	Autoencodeurs, Isolation Forest	Imiter le rythme du trafic normal	
Okada (2025)	MLP	Modifier le TCP aux points clés	
Mao (2025)	MLP, CNN, ResCNN, LSTM, attention	Tromper plusieurs modèles à la fois	
Zhang (2022)	MLP, CNN, C-LSTM	Interroger le détecteur, ajuster	
Venturi (2024)	E-GraphSAGE, LineGraphSAGE	Ajouter des communications	
Venkatesan (2021)	SVM	Ajouter des exemples mal étiquetés	
Das (2025)	SVM, Random Forest, MLP	Changer les étiquettes	
Alhussien (2023)	Régression logistique	Polluer une base de données externe	**Nouveau**
PCAP-Backdoor (2026)	DNN, CNN	Cacher un motif dans des paquets	**Nouveau**
Ma (2025)	AdaBoost, GBDT, XGBoost, MLP, CNN, RNN	Motif ajouté par le prestataire	**Nouveau**

Le site : construire une topologie réseau

Une interface web inspirée de Packet Tracer.

- Ajouter et déplacer des équipements par glisser-déposer.
- Relier les équipements aux switches avec l'outil câble.
- Modifier les paramètres réseau dans le panneau de propriétés.
- Vérifier la cohérence de la topologie avant le déploiement.

Exécution et observation : fonctions implémentées

Le code du prototype comprend :

- Le déploiement de la topologie sous forme de conteneurs et de réseaux Docker.
- Un terminal pour les PC et les serveurs démarrés.
- La capture des paquets sur une interface d'équipement reliée à un switch.
- L'affichage des paquets, des filtres et un export PCAP.

La détection ML et l'exécution des attaques restent à intégrer.

Démonstration du prototype

Intégration du ML-NIDS : travail restant

1. **Choisir et reproduire une référence :** fixer le modèle, l'extracteur et les données de départ.
2. **Relier la sonde au détecteur :** transformer le trafic capturé en caractéristiques et obtenir un score ou une décision.
3. **Intégrer une attaque compatible :** préciser les accès nécessaires et les limites de modification du trafic.
4. **Afficher la comparaison :** trafic de référence, trafic transformé et résultat de détection.

Premier objectif proposé : une expérience reproductible de bout en bout sur une topologie simple.

Questions pour Grégory Blanc

1. **Comparaison avant / après :** quelles informations afficher pour rendre l'effet de l'attaque facile à comprendre ?
2. **Représentation :** privilégier une vue du réseau, une chronologie des échanges ou des graphiques de résultats ?
3. **Déroulement :** montrer l'attaque en direct, étape par étape, ou présenter un bilan comparatif à la fin ?
4. **Message pédagogique :** que doit retenir l'utilisateur après avoir vu une démonstration ?